

SkyGuard As-Built

The brief, the mathematics, and what is actually running.

The blueprint describes an intended system and carries the reasoning that got us there. This document describes the **built** one, and shows the arithmetic underneath it in full — every formula with a worked example you can check by hand. Where the two documents disagree, this one is right: it was written by reading the code and querying the deployed service. Every number came from `/api/summary`, a model report, or a calculation reproduced from the source.

00 The whole thing, in one screen

Read this and you have it. Everything after is the working.

QUESTION	ANSWER
What does it detect?	A station drifting away from its neighbours, using only temperature, pressure and humidity. Weather is shared and cancels in the difference; a fault belongs to one instrument and does not.
How, in one line?	Remove each station's own hourly climatology, subtract the median of its neighbours, divide by a trailing lagged rolling MAD, band at 6σ and 8σ , and require three consecutive steps before opening an episode.
How accurate?	57% recall, 38% precision — 72% recall once dropouts, which have no residual to difference, are excluded. Against injected faults whose amplitudes deliberately run below what a 30-day record can support.
Who decides to dispatch?	Three deterministic agents — data quality, hardware health, weather-or-fault — and an adjudicator with a fixed rule order. Not an LLM, and not a learned model.

QUESTION	ANSWER
How is it explained?	Exact Shapley over the three agents (all eight coalitions, so contributions sum to the verdict), a counterfactual per agent, and the adjudicator's own walk shown step by step.
Is any of it ML?	No. There is no model in the decision path. The intelligence is in the multi-agent arbitration and the spatial statistics.
Is the data real?	Station <i>locations</i> are real IMD sites. Readings are generated, and stay generated through the finals — what is demonstrated is the pipeline that judges them.
Does hardware exist?	An ESP32 sketch that runs end to end in Wokwi, TLS to the deployed service, through the same ingest door the simulated network uses. Never run on a physical board.
Biggest gap?	The explanation is a fixed decision tree, not an autonomous investigation. See §18 and §26.

CONTENTS

00 In one screen

I • THE BRIEF

01 The problem statement

02 How it is scored

03 End-to-end workflow

II • THE MATHEMATICS

04 The physics you have

05 Why three parameters

06 Step 1 — anomaly

07 Step 2 — neighbours

08 Step 3 — sigma from MAD

09 Step 4 — z and the bands

10 Two passes

11 What it cannot see

12 Alerts as episodes

III • THE DATA

13 The network we made



- 14 Fault models
- 15 Real vs. generated
- 16 How accuracy is measured
- 17 Measured results
- 18 Explainability, as built

IV · AS-BUILT

- 19 Shape of the system
- 20 The live node
- 21 Ingest and storage
- 22 Module inventory
- 23 API surface
- 24 Frontend
- 25 Deployment
- 26 Not built
- 27 Traps already paid for
- 28 Running it

PART I – THE BRIEF

01 The problem statement

Detect abnormal, inconsistent or faulty observations from Automatic Weather Stations in real time, using **only** Temperature (°C), Atmospheric Pressure (hPa) and Relative Humidity (%). Distinguish genuine meteorological events from sensor/data anomalies while minimising false alarms and enabling scalable deployment across large networks.

REQUIRED	DETAIL
Objectives	Real-time detection · identify sensor faults, spikes, frozen values, communication errors · learn normal temporal and seasonal patterns · multivariate consistency analysis · confidence scores and explainable reasoning · predict sensor degradation and maintenance needs · optionally suggest corrected values.
Outputs	Real-time alerts · severity and confidence scores · root-cause classification · visualisation dashboard · sensor health status · corrected data (optional).

REQUIRED	DETAIL
Deliverable	"Fully executable code with example usage and a document explaining various use cases." Not a prototype and slides.
Evaluated on	Anomaly-injected data.

The one case that matters

The hard failure is drift. A probe that fails slowly reads a little wrong, then a little more, for weeks. No single reading is implausible, so no threshold fires. The failure is not the value — it is the divergence, and divergence is only visible against something that has not diverged. That is the idea the whole system rests on: **compare a station with its neighbours at the same moment**. Weather is shared between nearby stations and cancels in the difference; a fault belongs to one instrument and does not.

CONSEQUENCE

A station with fewer than three usable neighbours cannot be judged this way. The interface labels those NO NEIGHBOURS rather than guessing — visible on the Andaman & Nicobar stations in the station index.

02 How it is scored

CRITERION	WEIGHT	WHAT EARNS IT HERE
Innovation & Novelty	25%	A signature classifier that names <i>which</i> fault or <i>which</i> event
Detection Accuracy	20%	Neighbour-difference channel; POD-vs-amplitude curves
Real-Time Capability	15%	O(1) streaming state; published latency distributions
Explainability	10%	Uncertainty-propagated physics confidence; exact Shapley
Scalability	10%	Per-station auto-calibrated thresholds

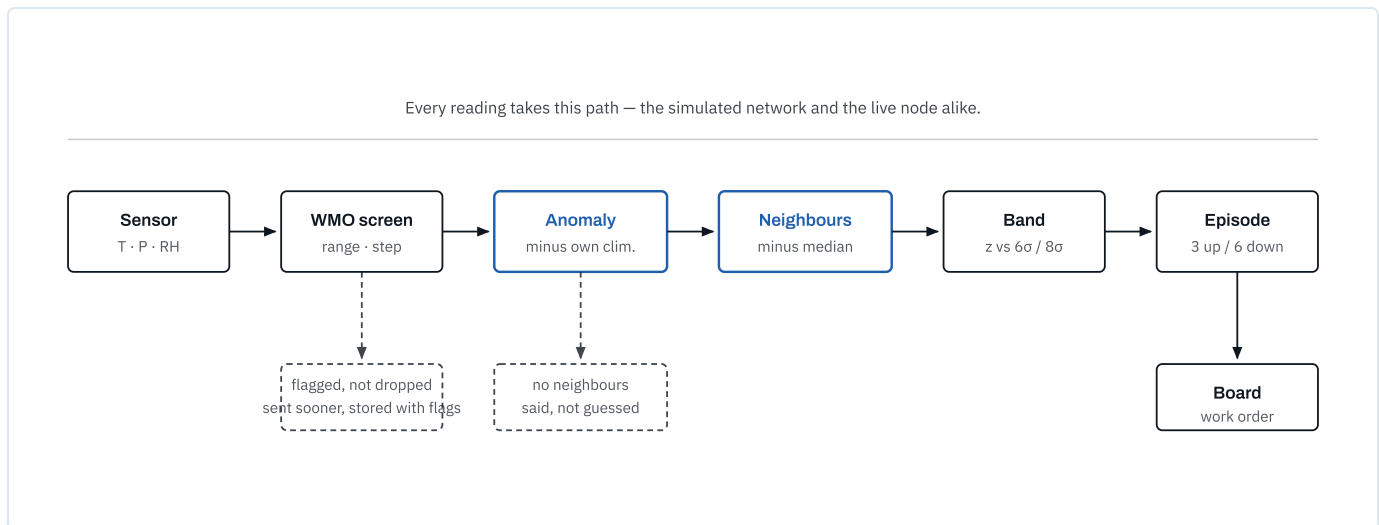
CRITERION	WEIGHT	WHAT EARNS IT HERE
Practical Deployability	10%	Integer-arithmetic edge tier
Visualization / UI	5%	Board, network, judgement screens
Energy Efficiency	5%	Measured uplink reduction

WHAT THIS DRIVES

Innovation (25) + Explainability (10) outweighs Accuracy (20). A classifier that *names* the fault and shows the physical reason scores on all three at once. A bare rarity score scores on one. That is why the pipeline ends in a named band and a named fault, not a probability.

03 End-to-end workflow

One pass, sensor to work order. Everything downstream of the sensor is arithmetic you can check by hand; the rest of Part II does exactly that.



Flags, never filters. A reading that fails the physics screen is not discarded — the flag is attached and the reading is sent, and on the node a flag actually brings the uplink *forward*: the firmware reads `if (flagged || due)`, so a suspect sample goes at once while a clean one waits for the heartbeat. The server does not take the node's word for it either: it recomputes the same screen, records `edge_screen_disagreement` when the two differ, and stores the reading either way. Discarding at the edge would leave nothing downstream able to reason

about the shape of a fault. The second branch is the one genuine refusal: a station with too few neighbours is reported unjudgeable rather than scored anyway.

PART II – THE MATHEMATICS

04 The physics you have

Three measured numbers. Everything else is derived from them by thermodynamics, and the derived quantities are more useful than the raw ones because they behave differently under weather than under failure.

Saturation vapour pressure — the Magnus form

Air can hold only so much water vapour before it condenses, and that ceiling rises steeply with temperature. The Magnus formula gives that ceiling:

$$e_s(T) = 6.112 \cdot \exp\left(\frac{a \cdot T}{b + T}\right) \text{ hPa} \quad \begin{array}{l} a = 17.62, \quad b = 243.12 \text{ over water,} \\ T \geq 0 \text{ }^\circ\text{C} \\ a = 22.46, \quad b = 272.62 \text{ over ice, } T < 0 \text{ }^\circ\text{C} \end{array}$$

The two coefficient sets matter. Using the water set below freezing manufactures spurious violations at high-altitude and northern stations — the formula would claim the air is supersaturated when it is simply cold.

Actual vapour pressure and dew point

Relative humidity is a *percentage of that ceiling*, which is why RH alone tells you little: 60% at 30 °C is far more water than 60% at 5 °C. Multiply to get the actual amount, then invert to get the temperature at which that amount would saturate — the dew point.

$$e = (RH / 100) \cdot e_s(T) \text{ hPa} \quad \gamma = \ln(RH/100) + a \cdot T / (b + T) \quad T_d = b \cdot \gamma / (a - \gamma) \text{ }^\circ\text{C}$$

WORKED — $T = 30\text{ }^{\circ}\text{C}$, $RH = 60\%$, $P = 1000\text{ hPa}$

$$e_s = 6.112 \cdot \exp(17.62 \cdot 30 / (243.12 + 30)) \quad 42.3372\text{ hPa}$$

$$e = 0.60 \cdot 42.3372 \quad 25.4023\text{ hPa}$$

$$\gamma = \ln(0.60) + 17.62 \cdot 30 / 273.12 \quad 1.4245$$

$$T_d = 243.12 \cdot 1.4245 / (17.62 - 1.4245) \quad 21.39\text{ }^{\circ}\text{C}$$

Dew point **21.39 °C**

Sanity check by eye: the air is at 30 °C and would need to cool to about 21 °C before dew forms. That is a humid but ordinary tropical afternoon. If the formula had returned 35 °C — a dew point above the air temperature — that would mean RH above 100%, which is the same statement, not a second independent check.

Specific humidity — this is what pressure is for

Specific humidity is the mass of water vapour per mass of air, in g/kg. Unlike RH, it does **not** change when the air simply warms or cools.

$$q = 0.622 \cdot e / (P - 0.378 \cdot e) \cdot 1000\text{ g/kg}$$

WORKED — SAME AIR PARCEL

$$0.622 \cdot 25.4023 \quad 15.8002$$

$$P - 0.378 \cdot e = 1000 - 9.602 \quad 990.398$$

$$q = 15.8002 / 990.398 \cdot 1000 \quad 15.95\text{ g/kg}$$

Specific humidity **15.95 g/kg**

Why this is the useful variable: **q is conserved under dry adiabatic processes and under ordinary diurnal heating**. The sun warms the air, RH falls, and q does not move. So an RH sensor that drifts breaks q, whereas genuine advection — a moist air mass arriving — moves q coherently and moves it at the neighbours too. That is a physical discriminator

between fault and weather, available from exactly the three parameters the problem statement allows.

Pressure must be reduced to sea level before comparison

Pressure falls roughly 1 hPa per 8–9 m of altitude. Comparing raw station pressure between neighbours at different elevations is meaningless.

$$P_{\text{MSL}} = P_{\text{station}} \cdot \exp\left(\frac{h}{29.271 \cdot (T + 273.15)} \right)$$

h in metres

WORKED – TWO NEIGHBOURS, 950 HPA EACH, T = 25 °C

Station A at 500 m	1006.02 hPa
Station B at 600 m	1017.61 hPa
Apparent difference if reduction is skipped	11.59 hPa

Eleven and a half hectopascals of pure artefact from 100 m of hill, against a genuine synoptic pressure anomaly that is often under 5 hPa. Skip this step and every hill station looks broken.

05 Why three parameters are enough

The constraint looks severe until you notice that T, P and RH are not three independent numbers — they are three views of one thermodynamic state, tied together by the relations above. That gives two independent kinds of evidence:

EVIDENCE	QUESTION IT ANSWERS	CATCHES
Cross-channel coherence	Do T, P and RH tell a consistent story <i>at this station</i> ?	A single failed channel: RH drifts while T and P do not, so q breaks while the neighbours' q holds.
Spatial coherence	Does this station agree with its neighbours <i>at this moment</i> ?	Anything that moves one instrument and not the weather — drift, offset, freeze,

EVIDENCE	QUESTION IT ANSWERS	CATCHES
		spike.

The discriminator between a fault and a storm is not magnitude. A genuine squall line produces a large, fast pressure fall — and so does a failing barometer. What separates them is that the squall arrives at the neighbours too, a few minutes earlier or later, and keeps T, RH and P mutually consistent while it does. The fault does neither. **Magnitude is not the signal; coherence is.**

06 Step 1 — anomaly, before anything else

The naive method is to subtract a neighbour's reading from this station's reading. Measured on this network, that flagged **49% of healthy stations**.

The reason is that a station's reading contains its own permanent identity. A mast at 4,196 m is colder than neighbours at 200 m, and — crucially — its *diurnal shape* differs too, because thin air warms and cools faster. So the raw difference swings every single day for reasons that have nothing to do with any instrument.

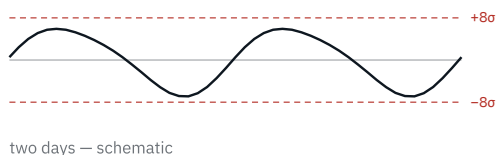
The fix is to remove each station's own clean-period climatology by hour of day first:

```
# hour of day for every step h(t) = (t · 15 min / 60) mod 24 # mean of
each hour over the clean period, per station s clim[h, s] = mean{ x(t, s)
: t < clean_steps, h(t) = h } anomaly a(t, s) = x(t, s) - clim[h(t), s]
```

Now every station's anomaly is centred near zero regardless of altitude, and shares the same units and roughly the same daily shape. What remains in `a` is weather plus faults — which is exactly what the difference is meant to compare.

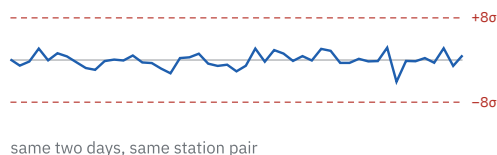
Raw difference

station minus neighbour, no anomaly step
crosses the band twice a day, every day



Anomaly difference

each station's own hourly climatology removed
quiet, and a real fault would stand out



Same station pair, same two days, drawn schematically. The left panel is why differencing raw values flagged 49% of healthy stations: the excursion is the daily cycle of an altitude difference, not a fault.

07 Step 2 — difference against the neighbours

For each station, take the **median** anomaly of its neighbours at the same timestep and subtract it:

```

$$r(t, s) = a(t, s) - \text{median}\{ a(t, j) : j \in N(s) \}$$

$$N(s) = \text{neighbours of } s;$$


requires  $|N(s)| \geq 3$ , else the station is not scored


```

Median, not mean, on purpose: the median of five neighbours is unmoved by one of them being broken, while the mean is dragged by it. This is the cheapest robustness in the whole pipeline.

WORKED — ONE TIMESTEP, TEMPERATURE ANOMALIES IN °C

Neighbour anomalies	+0.80 · +0.92 · +0.71 · +1.08 · +0.85
Their median	+0.85
Healthy station reads	+0.90
residual $r = 0.90 - 0.85$	+0.05
Same station with a +2.0 °C offset fault	+2.90

residual $r = 2.90 - 0.85$

+2.05

Read the healthy row carefully: every station in the region is about 0.85 °C warmer than its own climatology — a genuine warm day. The *weather cancelled completely*, leaving 0.05. The faulty row keeps almost the entire 2.0 °C of the fault. That contrast is the entire method.

A second, independent question: did the region move at all?

The residual above says this station is out of step with its neighbours. It says **nothing** about whether the neighbours were themselves doing something unusual at the time — and that is the difference between a drifting instrument in still conditions and a real front that this station merely exaggerated.

So a separate quantity is computed over the flagged window: the **neighbour median anomaly**, standardised by its own quiet spread. It is not what the grade was computed from, which is what makes it evidence rather than a restatement.

```
# what the region itself did, over the flagged window [lo, hi) region(t)
= median{ a(t, j) : j ∈ N(s) } σregion = 1.4826 · median( |region -
median(region)| ) whole record zregion = median( region[lo:hi] ) / σregion
# agreement needs all three, not just the first moved = |zregion| ≥ 2.0
same way = sign(region_dev) = sign(station_dev) explains = |region_dev| ≥
0.4 · |station_dev|
```

The third clause is the one that matters. A region that moved 2σ while the station moved 30 does not excuse the station — that is weather *plus* a fault, and the fault is the part worth dispatching for.

WHAT THIS REPLACED, AND WHY IT WAS WRONG

The board previously asserted `neighbours_agree: false` on every simulated row, reasoning that a flagged station is by definition one its neighbours disagree with. That handed the context agent its own conclusion: it could never veto, and its exact Shapley value across the whole board was structurally zero — three agents, one of which could not affect any outcome.

MEASURED AGAINST THE INJECTED TRUTH — ALL 60 FLAGGED ROWS

Vetoed issued (“this is weather”)	9
Correct — no fault had been injected	8
Veto precision	89%
False alarms removed	8 of 46 — 17%
Real faults wrongly vetoed	1

The single miss is a dropout, and the adjudicator alarms on it anyway because hardware is checked *before* the weather veto — a dead link is a fault in any weather. That ordering was chosen for a different reason and happens to cover this case, which is worth knowing rather than claiming as design.

08 Step 3 — how big is "big"? Sigma from the MAD

A residual of 2.05 °C means nothing until you know the ordinary spread of that station's residuals. A calm inland site might sit within ± 0.2 °C; a coastal site in monsoon might routinely swing ± 1.5 °C. So each station gets its own scale, estimated robustly.

What the MAD is, and where 1.4826 comes from

The Median Absolute Deviation is the median of the distances from the median. Unlike the standard deviation it is not inflated by a handful of extreme values — which matters enormously here, because the extreme values are the faults we are trying to measure *around*.

$$m = \text{median}(x) \quad \text{MAD} = \text{median}(|x - m|) \quad \sigma = 1.4826 \cdot \text{MAD}$$

The constant is not arbitrary. For data that really is Gaussian, the MAD comes out at about 0.6745σ — that is the 75th percentile of the standard normal. Dividing by 0.6745 gives 1.4826, so the scaled MAD is directly comparable to a standard deviation. **You can check it: $1 / 0.6745 = 1.4826$.**

WORKED — TEN RESIDUALS, °C

r	0.12 -0.05 0.31 -0.22 0.08 0.44 -0.11 0.02 0.19 -0.30
median m	0.050
r - m	0.07 0.10 0.26 0.27 0.03 0.39 0.16 0.03 0.14 0.35
median of those = MAD	0.150
$\sigma = 1.4826 \cdot 0.150$	0.2224 °C

Shrinkage toward the network

One window of one station is a noisy estimate, and the stations it underestimates are exactly the ones that will cry wolf. So each station's own estimate is pulled halfway toward the network-wide median of all stations' MADs:

```
pooled = medianall stations( MAD ) sd = max( 0.5·MAD + 0.5·pooled , 1e-6 )
```

WORKED — CONTINUING, WITH A NETWORK POOLED MAD OF 0.42

own estimate	0.2224
pooled	0.4200
$sd = 0.5 \cdot 0.2224 + 0.5 \cdot 0.4200$	0.3212 °C

The window is trailing, lagged, and re-estimated on a stride

This is the correction that mattered most in the whole project.

PARAMETER	VALUE	WHY
SIGMA_WIN	15 days	Long enough for four independent samples given a 2.5-day weather correlation time
SIGMA_LAG	1 day	Holds the reference behind the point being scored, so a slow drift cannot hide inside its own sigma

PARAMETER	VALUE	WHY
SIGMA_STRIDE	6 hours	The spread of a weather residual does not move in six hours; exact rolling MAD buys nothing for the cost
MIN_BASE	5 days	Never estimate a scale from less than this

THE BUG THIS REPLACED – MEASURED

Sigma used to be the MAD of the first half of the record, applied to all of it. On a network with *no faults injected at all*:

```
baseline window |z| p95 = 2.78
scoring period |z| p95 = 4.47 ← inflated 1.61×
false alarms at 4σ: 7.6% of station-hours
```

Two mistakes, both the same mistake. The estimate was **in-sample** — fitted to the window it was then evaluated on — and it was **fixed**, so a month of changing weather was scored against a fortnight of one regime. A threshold of 4 was really about 2.5 by the end, and that is the entire false-alarm rate. It is also the only version a deployed system could run: at 09:00 you have yesterday, not next week.

Self-masking is the reason for the one-day lag. A drift that has already entered the reference window inflates sigma and thereby hides itself. Holding the reference a day behind buys the detector the beginning of the fault before the fault can start defending itself. This does not solve self-masking for a fault longer than the window — nothing that uses the station's own history can — and that limit is stated wherever the recall figure appears.

09 Step 4 — the z-score and the two bands

```
z(t, s) = ( r(t, s) - m ) / sd |z| ≥ 6 → WATCH WATCH_SIGMA = 6.0 |z| ≥ 8 →
FAULT FAULT_SIGMA = 8.0
```

WORKED – CARRYING BOTH STATIONS THROUGH, SD = 0.3212, M = 0.050

Healthy: $z = (0.050 - 0.050) / 0.3212$

0.00 → ok

Offset fault: $z = (2.050 - 0.050) / 0.3212$

6.23 → WATCH

Six sigma sounds extreme against the usual "3 sigma" intuition, and it is deliberately so. Three sigma on a Gaussian is roughly one exceedance in 370 samples; across 344 stations at 96 steps a day, that is around 90 false flags *per day* from noise alone. Residual distributions also have heavier tails than a Gaussian, which makes it worse. The bands were calibrated empirically against a fault-free control run rather than assumed from theory, which is why they are 6 and 8 rather than 3.

Both constants are named once, in `simulate/faults_and_export.py`, and imported everywhere else. They were previously written in two places, and the map and the alert list disagreed about what was flagged.

10 Two passes — a faulty neighbour poisons the reference

Measured on the first version: healthy stations had $|z|_{p95} = 2.00$ in the training window — exactly what the MAD predicts — and `4.33` afterwards, which flagged half of them. The residual was not drifting. The *reference* was.

A frozen or stepped station still sits inside its neighbours' median, and where a station has only four or five neighbours, one bad member moves that median enough to condemn the rest.

```
pass 1 z1 = grade(all stations) suspect if a station is beyond 3σ for more
than 20% of the record suspect(s) = meant( maxchannels |z1(t,s)| ≥ 3 ) >
0.20 pass 2 z = grade(all stations, excluding suspects from everyone's
N(s))
```

Note what is *not* consulted: ground truth. The second pass uses only the first pass's own output, which is what a deployed system would have.

11 What the detector can and cannot see

The bands and sigma together imply a hard detection floor, and it is worth stating in physical units rather than leaving it implicit.

A drift must accumulate past $6 \cdot \text{sd}$ before it reaches WATCH. With the worked example's sd of 0.3212°C :

DRIFT RATE	REACHES WATCH ($6\sigma = 1.927^\circ\text{C}$)	REACHES FAULT ($8\sigma = 2.570^\circ\text{C}$)
$0.15^\circ\text{C} / \text{day}$	12.8 days	17.1 days
$0.05^\circ\text{C} / \text{day}$	38.5 days	51.4 days
$0.0642^\circ\text{C} / \text{day}$	30 days – the whole record	never, in this record

SAY THIS OUT LOUD

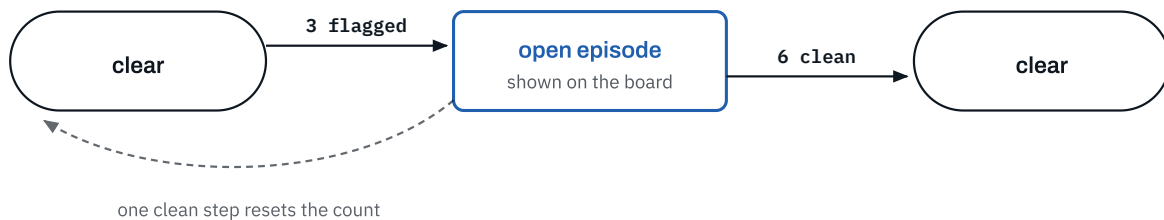
Against a 30-day record and this sigma, **a drift slower than roughly $0.064^\circ\text{C}/\text{day}$ cannot reach the WATCH band at all**, and slower than $0.086^\circ\text{C}/\text{day}$ cannot reach FAULT. It is not that the detector is tuned badly — the evidence is not there yet. A longer record moves this floor down proportionally.

This is also the honest answer to "why is recall 57% and not 95%": the injected drift amplitudes deliberately go down to $0.005^\circ\text{C}/\text{day}$, far below anything a month of data can support. A detector reporting 99% recall on this set would be reporting a bug.

The second blind spot is **dropout**. A station that stops reporting has no value to difference, so it produces no residual and no z at all. It is **NO DATA**, not a fault — which is why the results carry a second recall figure with dropouts excluded.

12 Alerts as episodes, not flags

A grade is recomputed every step with no memory of the last one. Thresholding it directly produced **2,484 flagged runs, half of them a single step long**. An alert therefore has a beginning and an end.



`RAISE_AFTER = 3` , `CLEAR_AFTER = 6` in `web/src/lib/alerts.ts` . Clearing is deliberately slower than raising — otherwise a drifting sensor that wobbles across the band closes its own alert.

PART III – THE DATA

13 The network we made

The generator places stations at **real IMD coordinates** and synthesises a physically coherent weather field over them, then injects faults into a known subset. Because the truth is known by construction, recall and precision can be computed exactly — which is impossible with real IMD data, since no labelled fault list is published.

PARAMETER	VALUE
Stations	344
States & territories	33
Record length	30 days
Step	15 min
Steps per station	2,880
Channels	temp · rh · pres

PARAMETER	VALUE
Record starts	2026-07-25
Stations with an injected fault	28

Station assignment to states is by point-in-polygon against a simplified boundary, with a nearest-polygon fallback for coastal and island sites that fall just outside their polygon. The fallback ranks by *state*, not by polygon — ranking by polygon made an island's runner-up the same state and the ambiguity test always passed.

14 Fault models — exactly what is injected

Seven fault types, each with an amplitude range and duration drawn log-uniformly. Amplitudes deliberately span from *trivially detectable* down to *physically impossible to detect in this record*, so that the result is a detection curve rather than a single flattering number.

TYPE	WHAT HAPPENS TO THE VALUE	WEIGHT
spike	isolated points get $\pm \text{amp}$; only those points are marked touched	0.20
frozen	$v[:] = v[0]$ — the last good value repeats exactly	0.18
calibration_drift	$v += \text{sign} \cdot \text{amp} \cdot t / 24$ — amp is per day	0.17
step_offset	$v += \text{sign} \cdot \text{amp}$ — a constant shift	0.15
dropout	$v[:] = \text{NaN}$ — no observation. Not zero, never imputed	0.12
noise_burst	$v += N(0, \text{amp})$ — variance inflation only	0.10
saturation	pinned at a rail, plus the jitter a stuck ADC still shows	0.08

Amplitude ranges

TYPE	TEMP	RH	PRES
spike	0.5 – 15.0 °C	3 – 60 %	0.5 – 20 hPa
step_offset	0.2 – 5.0 °C	1 – 20 %	0.2 – 8 hPa
noise_burst	0.2 – 4.0 °C	1 – 15 %	0.2 – 5 hPa
calibration_drift	0.005 – 0.15 /day	0.02 – 0.6 /day	0.005 – 0.2 /day

Durations run from 1–3 hours for a spike to 240–1440 hours for a calibration drift. Weights follow real AWS failure logs, where drift and frozen values dominate.

WHY THE RANGES GO DOWN SO LOW

"A detector tuned against 5 K spikes will report 99% recall and miss every real 0.4 K drift." The point of the low end is to make the evaluation produce a *probability-of-detection versus amplitude* curve, which tells you the smallest fault the system can actually find — the number an operator needs — instead of one recall figure that hides it.

One subtlety worth knowing: for spikes, only the displaced samples are marked as touched. Marking the whole window depressed every spike metric in the repo, because a three-hour spike window contains mostly untouched samples that were being scored against a fault that never reached them.

15 Real vs. generated

The first question any reviewer asks, answered before anything else. **Station locations are real IMD sites. The readings are generated.** They will stay generated through the finals — what is being demonstrated is the pipeline that judges them.

PLANE	LOCATIONS	READINGS	FAULT LABELS
Simulated network	real IMD sites	generated	injected — truth known
Live node	real — Gandhinagar	generated, arriving now	injected on demand

PLANE	LOCATIONS	READINGS	FAULT LABELS
WDQMS	real IMD stations	real departures	none published

Every accuracy figure in this document rests on faults we injected ourselves. IMD publishes station locations, not a labelled fault list, so there is no real archive to validate against. What injected truth buys instead is exactness: recall and precision here are computed against a ground truth known by construction, not estimated.

16 How accuracy is measured

Every accuracy claim in this project is one of the definitions below. They are not interchangeable, and quoting the wrong one is the easiest way to sound better than you are.

Precision and recall

recall = faults found / faults present did we miss anything? **precision** = true alarms / all alarms is the queue worth working?

WORKED – THE ACTUAL NETWORK RESULT

Stations with an injected fault	28
Stations the detector alerted on	42
Of those, genuinely faulty ($0.38 \cdot 42$)	16
recall = $16 / 28$	0.57
precision = $16 / 42$	0.38

Read together: of 28 real faults we find 16, and to get those 16 an engineer is asked to look at 42 stations. Twenty-six of those visits find nothing. That is the real cost, and it is the number a maintenance planner cares about far more than recall.

Event recall vs. point recall

These answer different questions and usually differ a lot.

MEASURE	COUNTS	TELLS YOU
Point recall	individual flagged samples inside faults	whether you catch the <i>bulk</i> of a fault
Event recall	an event is a hit if any sample in it is flagged	whether you catch the <i>onset</i> at all

High event recall with low point recall means the system notices faults but does not track them; the reverse means it is good at long obvious faults and misses short ones. The gap between the two is the diagnostic.

THE OPERATING POINT IS A CHOICE, NOT A RESULT

This same detector reaches **86% recall at 15% precision** if the bands are loosened. Neither setting is "the accuracy". We publish 57/38 because a queue nobody trusts is worse than a shorter one — but the trade is a dial, and an operator with more field crews should turn it.

17 Measured results

From `/api/summary` on the deployed service — 28 faults injected into a 344-station network, with the grader given the readings and nothing else.

57% RECALL of injected faults found	38% PRECISION of alerting stations	72% RECALL EXCL. DROPOUTS silence has no residual	73 ALERT EPISODES 42 stations alerting
--	---	---	---

The dropout-excluded figure jumps from 57% to 72% because a station that stops reporting produces nothing to difference. Counting those as detection failures blames the detector for a communications outage, so both numbers are published and the reason is stated with them.

THE HONEST HEADLINE

There is no machine-learned model in the decision path. The three agents are deterministic evaluators, the adjudicator is a fixed rule order, and nothing trains or updates online. Calling the panel “agents” describes the architecture — independent specialists, one arbiter, every verdict inspectable — not an LLM in a loop.

Why not SHAP

The problem statement names SHAP/LIME as preferable. Both attribute a *model's* prediction to *its features*; applied to a fixed threshold they return the threshold. But SHAP is an *approximation* of the Shapley value from cooperative game theory, and it approximates only because a model has hundreds of features and 2^n coalitions is impossible to enumerate.

We have three agents. 2^3 is eight. Every coalition is evaluated, so this is the exact Shapley value — the original theorem, not a sampled estimate — for three adjudications more than the verdict itself.

```
# payout: how hard a coalition pushes toward a visit v(S) = escalation(
  adjudicate with only the agents in S allowed to speak ) normal → 0.0
  warning → 0.5 critical → 1.0  $\phi_i = \sum_S ( |S|! \cdot (n-|S|-1)! / n! ) \cdot [ v(S \cup \{i\}) - v(S) ]$ 
  weights 1/3, 1/6, 1/3
```

An absent agent is **silenced**: it returns `unknown` with no confidence, which is what the adjudicator already does with an agent that cannot see its evidence, so absence needed no special case. The empty coalition scores zero — all three silent reaches “not enough readings yet” — which makes the contributions sum *exactly* to the escalation, the efficiency axiom a force plot usually only claims.

WORKED — MANDI, RELATIVE HUMIDITY, FROM THE DEPLOYED SERVICE

Data quality — reading outside the band

$\phi = +0.25$

Hardware health — nothing to look at	$\phi = 0.00$
Weather or fault? — 15 neighbours moved 3.7σ	$\phi = -0.25$
Sum = escalation as issued → MONITOR	0.00

Read the third row: the weather check did not abstain, it argued *against* dispatch by exactly as much as data quality argued for it, and won. A label saying “data quality carried this” cannot express that; a signed number can.

Each contribution also ships the action the panel would reach without that agent, computed by re-running the adjudicator with it silenced. **“Silence hardware and this becomes CALIBRATE”** is the half people act on.

The decision trace

The adjudicator records every check it reached, in order, with the answer it got, shown on the board in a collapsed disclosure. The checks that did *not* fire carry most of the value: “send someone” means little, while “hardware is fine, the neighbours moved 0.0σ , the readings are 8σ out” is a case. Answers are three-valued — a check whose evidence is missing records `n/a`, because folding that into “no” reads as a finding when it is an absence.

A LIMIT WORTH STATING BEFORE SOMEONE ELSE DOES

The trace walks a **fixed decision tree**: across all sixty flagged board rows it takes exactly **two distinct paths** from six question strings. It is an honest printout of the rules and it is not an autonomous investigation — nothing chooses what to examine next based on what it just found. See §26.

Is it really three agents?

Gathering the same Shapley values across all 60 flagged rows answers what no single row can — whether an agent is an ornament.

AGENT	DECIDED	MEAN ϕ	READING
Data quality	60 / 60	+0.63	Carries almost every call here

AGENT	DECIDED	MEAN Φ	READING
Hardware health	0 / 60	0.00	An ornament on this network
Weather or fault?	9 / 60	-0.05	Only ever argues <i>against</i> a visit

Grouped by the fault that was *injected* — which the panel is never shown — the same numbers become a capability map, where an empty column is a fault class nothing detects. Counts ship with every cell and thin cells are drawn back, because six fault kinds across fourteen injected faults means some cells rest on two rows.

A FINDING, AND A FIX THAT DID NOT WORK

Hardware scores zero on all sixty rows because the simulated export carries no housekeeping channels, so it has nothing to read. It fires correctly on the live node, which sends them.

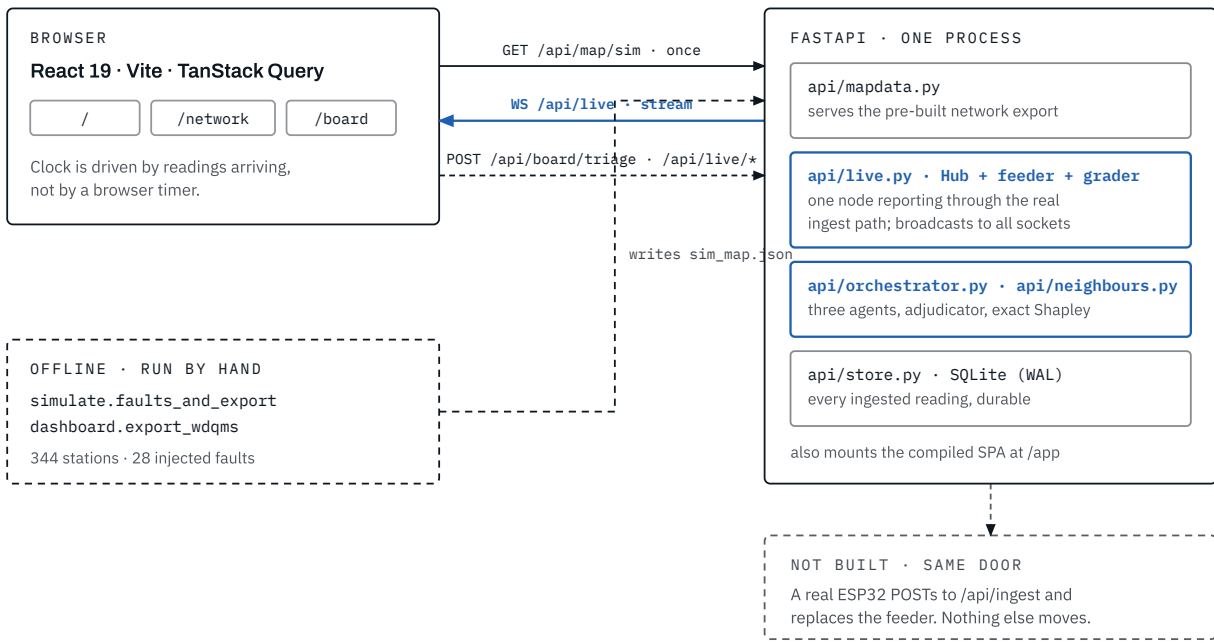
A stuck probe should be visible in the readings, so flat fraction was tried two ways: consecutive unchanged samples (`stuck` scored 0.24 and 0.50 against a 0.02–0.14 background) and longest unchanged run (`stuck` scored **0.004**, below several healthy stations). Neither separates at this sampling and quantisation, so it is not computed. Lowering the threshold until it fired would be fitting the demonstration rather than detecting anything.

PART IV — AS-BUILT

19 Shape of the system

One FastAPI process serves everything: the JSON exports, the live WebSocket, the triage panel, and the compiled React app itself. There is no separate frontend host and no database server.

The panel is the only place the decision rules exist. The board sends the evidence it is allowed to see and renders whatever comes back, holding no judgement of its own — reimplementing “is this a fault” in the browser to save a request is how two screens end up disagreeing about the same sensor.



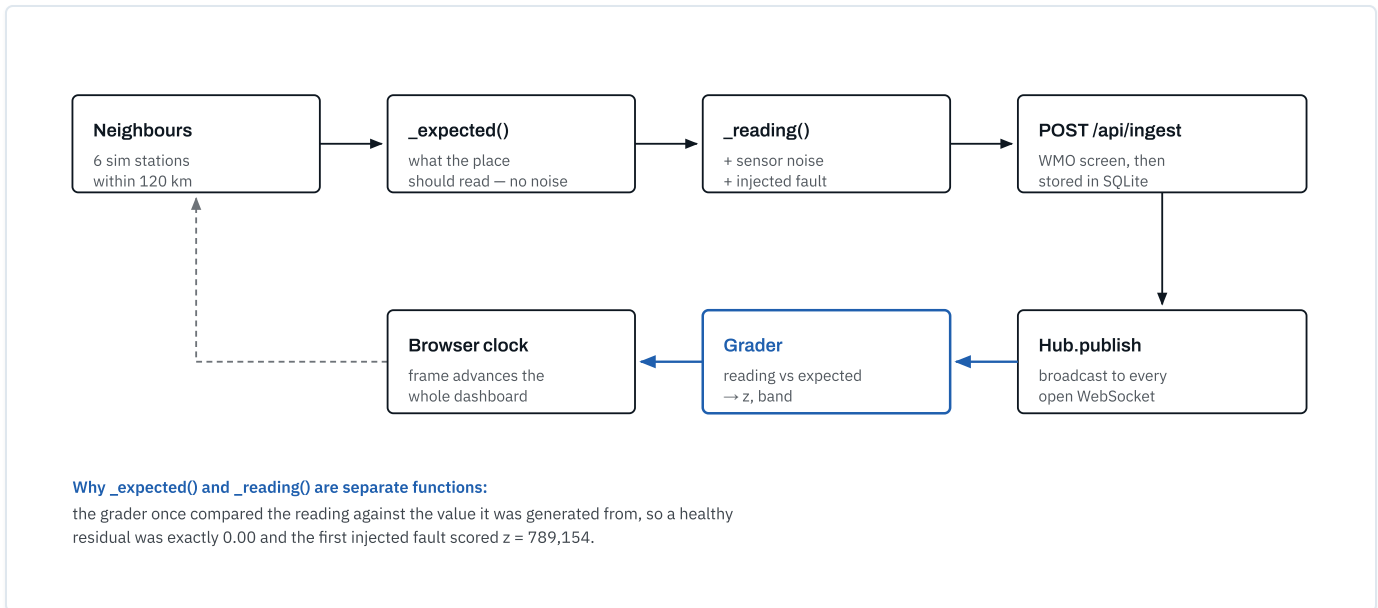
The browser holds three channels to one process. The dashed box on the left is not a service — it is two commands a developer runs to regenerate the JSON the API serves. The dashed box at the bottom is the only piece that hardware changes.

20 The live node

A stand-in for hardware, not a simulation shortcut. It posts through the same `/api/ingest` door an ESP32 would use, and it is graded online, against real neighbours, by the same sigma bands the map uses.

FIELD	VALUE
Station id / name	ESP32-01 · SKYGUARD ESP32
Location	Gandhinagar, Gujarat · 23.2156, 72.6369 · 81 m
Neighbours used	6 simulated stations within 120 km
Simulated step	30 min per reading

FIELD	VALUE
Baseline before judging	40 readings – reports "learning" until then
Sensor noise added	T 0.25 °C · RH 1.2 % · P 0.15 hPa
Injectable faults	drift · offset · stuck · spike · dropout



The node drives the clock. Nothing starts by itself — Play starts it, and only Play.

BEHAVIOUR WORTH KNOWING

Pause and resume continue the record rather than restarting it, because a station that stops transmitting and starts again does not begin its record over. Changing Speed retunes the running node in place via `/api/live/rate` — it does not restart it, which would reset the clock to day 0.

21 Ingest and storage

`POST /api/ingest` is the single door into the system. It applies the WMO-No. 8 Annex 1.A physics screen (`physics/wmo_limits.py`), then writes the reading to SQLite in WAL mode via `api/store.py`. Readings survive a restart; `/api/ingest/history` and `/api/ingest/recent` read them back.

It screens but does not reject. `physics_screen()` is recomputed server-side rather than trusted — a node with a corrupted screen is one of the faults being looked for — and where the node's flags and the server's disagree, `edge_screen_disagreement` is added to the record. Every reading is stored with whatever flags it earned. This matters for the demo's credibility: in the real world the data is stored, so it is stored here too.

The simulated network does not come through here. It is a pre-built export the panel reads directly, so nothing about it is ingested or stored — the two sources meet at the triage panel rather than at the door. Making the simulation travel the same path is deferred work, listed in §26.

22 Module inventory

What is in the serving path, what is offline tooling, and what is research code that no longer runs in production. `SERVED` means the API imports it at runtime.

MODULE	STATUS	ROLE
<code>api/main.py</code>	<code>SERVED</code>	App, routers, SPA mount, GZip
<code>api/live.py</code>	<code>SERVED</code>	Hub, WebSocket, feeder, online grader, fault injection — 699 lines
<code>api/ingest.py</code>	<code>SERVED</code>	The single ingest door; WMO screen
<code>api/store.py</code>	<code>SERVED</code>	SQLite WAL persistence
<code>api/orchestrator.py</code>	<code>SERVED</code>	Three agents, adjudicator, exact Shapley, decision trace — 702 lines
<code>api/neighbours.py</code>	<code>SERVED</code>	Measured regional movement and the comparison series — 364 lines
<code>api/engine.py</code>	<code>SERVED</code>	Station series and baselines
<code>api/mapdata.py</code>	<code>SERVED</code>	Serves the pre-built exports
<code>api/tiles.py</code>	<code>SERVED</code>	Map and boundary tiles

MODULE	STATUS	ROLE
physics/relations.py	SERVED	Magnus, dew point, q, MSL reduction — dependency-free, transliterates to C++
physics/wmo_limits.py	SERVED	WMO-No. 8 Annex 1.A limits
detect/ baseline · features · learned · belief · checkers	SERVED	Imported by the engine
health/state.py	RESEARCH	Sensor health state; not imported by the API
simulate/ network · faults_and_export	OFFLINE	Builds the 344-station network, injects faults, grades, exports
inject/faults.py	OFFLINE	The seven fault models of §14
dashboard/export*.py	OFFLINE	Writes the JSON the API serves
detect/ trust · allocate · supervised	RESEARCH	Not imported by the API
evaluation/ — 18 modules	RESEARCH	Offline experiments; only metrics.py is imported
firmware/skyguard_node	RUNS IN SIMULATION	ESP32 sketch. #define SENSOR_BME280 picks the sensor: BME280 on real hardware, DHT22 + potentiometer in Wokwi, which has no BME280 at all. Verified end to end in Wokwi — TLS uplink to the Space, housekeeping accepted, panel returns “check power and communications”. Never run on physical hardware.
firmware/virtual_node.py	OFFLINE	Same wire protocol in plain Python

GROUP	ENDPOINTS
Live node	WS /api/live · /api/live/replay · /stop · /rate · /seek · /fault · /standing · /station · /status
Ingest	POST /api/ingest · /api/ingest/history · /api/ingest/recent
Network data	/api/map · /api/map/{name} · /api/stations · /api/stations/{station}/series
Triage panel	POST /api/board/triage · /api/board/panel · /api/board/region · /api/board/behaviour
Node health	/api/node/{station}/health
Alerts	/api/alerts · /api/alerts/{id} · /api/baseline/{station} · /api/explain · /api/rescore
Tiles	/api/tiles/{ver}/{z}/{x}/{y}.png · /api/boundary/... · /api/tiles/status
Meta	/api/summary · /api/meta · /api/clock · /api/snapshot · /healthz · /readyz

`/api/summary` is the authoritative source for every headline number. Do not hardcode these figures anywhere — read them.

24 Frontend

React 19 + Vite + TypeScript, TanStack Query for server state, react-router for routing, Leaflet for the map. Compiled to `web/dist` and mounted by FastAPI at `/app`. Four routes, and no console — the old HTML console was retired.

Styling is **Tailwind v4** via `@tailwindcss/vite`, with shadcn-idiom primitives (`cva` + `cn`). The migration turned on one detail worth recording: the pre-existing stylesheets are imported into an explicitly declared `@layer legacy` that is listed *first*, so Tailwind utilities outrank them. Without that declaration unlayered legacy CSS wins every tie by specificity,

converted pages render as though nothing changed, and `bg-ink text-paper` comes out black on black.

```
web/src/
  routes/      HomeRoute · NetworkRoute · BoardRoute · TeamRoute
  lib/         useLive (WebSocket singleton) · alerts (episodes) · triage (panel client)
               sticky (sessionStorage state) · bands · field · sim · theme · title · clock
  api/         client · queries · types · mapTypes · sites
  components/  AgentPanel · PanelAttribution · DecisionTrace · PanelBehaviour
               RegionCompare · ExplainMath · StationChannels · BeliefChart
               Callout · Field · Badge · ThemeToggle · Async
  components/home/ SkyPanel · XaiLoop · EdgeTiers · DriftFigure · Reveal
```

The explainability surface is four components and they are deliberately separate:

`AgentPanel` shows the three opinions, `PanelAttribution` the exact Shapley values as a waterfall with a lane per agent, `DecisionTrace` the adjudicator's walk in a collapsed disclosure, and `PanelBehaviour` the same attribution gathered across the whole network. `RegionCompare` draws the station against its neighbours, which is the evidence the verdict rests on.

`lib/sticky.ts` keeps the network page's eleven controls in **sessionStorage**, not `localStorage`, on purpose: these are working settings that should survive navigating to the board and back but should not still be there tomorrow showing a colleague a view they never chose.

`lib/useLive.ts` is the other file worth reading before touching anything live: a module-level socket shared by every subscriber, with reconnect and backoff, holding `byFrame`, `grades`, `frame`, `pass`, `running` and `fault`. The dashboard clock is derived from `live.frame` — **if readings stop arriving, time stops on the page**. That coupling is deliberate, and it is also why several past bugs presented as "the Play button is broken".

25 Deployment

TARGET	LOCATION	NOTES
Live service	Dev-66/skyguard-api	Hugging Face Space, public. Serves API + SPA at <code>/app</code> .
Source	Devansh-66/skyguard	GitHub, branch <code>main</code>
SDK	Gradio on ZeroGPU	See the caveat below

TWO THINGS THAT WILL CONFUSE THE NEXT PERSON

ZeroGPU on a CPU-only project. A free account cannot create a Docker or cpu-basic Space — the API answers 402. ZeroGPU is the only free way to get a Python process, using the documented `no-op @spaces.GPU` pattern with all real work outside it. No GPU is ever requested and no quota is consumed.

One line holds the whole API up. The entrypoint launches Gradio with `prevent_thread_lock=True` , then `demo.app.mount("/", api)` . Delete that line and every API route 404s while the Space still reports RUNNING.

26 Not built

Stated plainly, so nobody rediscovers these as surprises.

GAP	STATE	DETAIL
An autonomous explanation trajectory	NOT BUILT	The largest gap. The decision trace is a walk through a fixed tree — two distinct paths across all sixty board rows, from six question strings. A trajectory chooses the next probe <i>because of</i> the previous answer, so different sensors produce genuinely different investigations. The probes largely exist; the controller that selects among them and the stopping rule do not.
A learned reference model	NOT BUILT	The place a model belongs is predicting what a station <i>should</i> have read, so the anomaly is the subtraction and SHAP over its features is honest. Designed, not trained. Until it exists there is no model in the decision path at all.

GAP	STATE	DETAIL
Faithfulness testing	NOT BUILT	Nothing verifies the explanations are not lying. We inject the faults, so “does the top-attributed factor match the injected cause?” is measurable here in a way it is not for most entrants. Not yet measured.
One door for both sources	DEFERRED	Agreed, and not yet done. The live node and a real ESP32 post through <code>/api/ingest</code> and are stored; the simulated network does neither. It is a pre-built export the panel reads directly, and <code>store.record()</code> is called only from <code>api/ingest.py</code> . Routing the simulation through the same door is the correct shape — one path, one screen, nothing special-cased — and the blueprint's system diagram already draws it that way. Until then the shared door is the <i>panel</i> , not <i>ingest</i> : both sources are judged by the same three agents, which is what actually stops two screens disagreeing about one sensor.
Real hardware	NOT BUILT	The firmware runs end to end in Wokwi against the deployed Space. It has never run on a physical ESP32.
Housekeeping on simulated rows	NOT BUILT	The export carries no supply, logger or link channels, so the hardware agent is silent across the whole simulated board. See §18 — deriving it from the readings was attempted and does not separate.
Authentication	NOT BUILT	Every endpoint is open, including the control POSTs.
Multi-node live tier	NOT BUILT	Exactly one live node exists; <code>_state</code> is a single global.
TimescaleDB / Postgres	NOT BUILT	The blueprint proposes it; storage today is one SQLite file.
Corrected-value output	NOT BUILT	Listed as optional in the problem statement; not attempted.
Automated tests	NOT BUILT	Verification is manual, through the browser and the API.

Each of these cost real debugging time. They are recorded so they cost it once.

TRAP	WHAT ACTUALLY HAPPENS
Differencing raw values	Flags 49% of healthy stations. Remove each station's hourly climatology first (§06).
Sigma fitted in-sample	Inflated 1.61×, 7.6% false alarms. It must be a trailing, lagged rolling MAD (§08).
Skipping the MSL reduction	100 m of elevation is ~11.6 hPa of pure artefact. Every hill station looks broken.
Magnus water coefficients below 0 °C	Manufactures supersaturation at cold and high stations. Select the coefficient set by temperature.
Marking a whole spike window as touched	Depresses every spike metric: most samples in the window were never displaced.
Mounted ASGI apps get no lifespan events	Startup hooks on the mounted FastAPI app never fire on the Space.
Awaiting inside a <code>finally</code> during cancellation	The <code>await</code> is cancelled immediately, so a "publish on shutdown" placed there never leaves the process.
The boundary GeoJSON is a bare <code>Feature</code>	Not a <code>FeatureCollection</code> — clipping is skipped silently if you assume otherwise.
A frame high-water mark	<code>Math.max</code> made the clock a ratchet: after a restart it could never go back, and froze permanently.
<code>pkill</code> on Windows	Does nothing. The old server keeps serving stale code while a second one fails to bind. Kill the port owner found via <code>netstat -ano</code> .
Unlayered legacy CSS beats Tailwind	Utilities lose every tie to un-layered stylesheets. Declare <code>@layer legacy, theme, base, components, utilities</code> first and import the old CSS into <code>legacy</code> , or converted pages render unconverted.

TRAP	WHAT ACTUALLY HAPPENS
Gating an entrance animation on opacity	A <i>paused</i> animation still applies its current keyframe, so a stalled or never-started entrance leaves content at opacity 0 permanently. Five sections shipped invisible. Animate transform only — the worst case is then content fourteen pixels low.
Wokwi has no BME280	The part does not exist in its library, so the I ² C bus is simply empty and the node looks mute. Use DHT22 behind a <code>#define</code> . Its serial pins are <code>TX / RX</code> , not <code>esp:TX0 / esp:RX0</code> , and a slide switch must ground pin 3 — grounding pin 1 boots with the link already cut.
Counting the wrong bits	<code>checks_passed</code> counted zero bits in the self-test mask and reported 1 when all 12 passed. It is <code>12 - popcount(mask)</code> .
Handing an agent its own conclusion	Asserting <code>neighbours_agree: false</code> on every flagged row made the context agent's Shapley value structurally zero. An agent that cannot change any outcome is an ornament, and only a global view reveals it.
Attribution ties on a vetoed row	Picking the carrier by largest $ \phi $ credits the agent that <i>lost</i> : +0.25 and −0.25 tie, and the verdict was “no visit”. The carrier is the agent whose absence would change the action.
Overlapping segments on one track	Drawn on a shared axis, two agents who disagree overlap and the one painted last hides the other — erasing exactly the case worth seeing. A lane per agent.
A strict <code>></code> on a shaded window	An episode shorter than one chart point collapses to zero width and never draws, on precisely the rows whose window matters most.

28 Running it

Serve locally

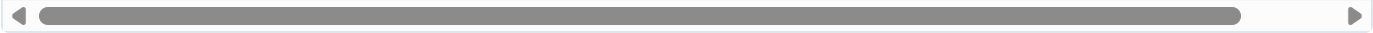
```
python -m uvicorn api.main:app --port 8000
# then open http://localhost:8000/app/network
```

Rebuild the frontend

```
cd web && npm run build      # writes web/dist, which FastAPI mounts at /app
```

Regenerate the network

```
python -m simulate.faults_and_export  # 344 stations, 28 faults -> dashboard/sim_map  
python -m dashboard.export_wdqms      # real WQMS departures
```



The node does not start on its own, locally or on the Space. Press Play, or `POST /api/live/replay?per_second=4` .